

P3440R1

Add `n_elements` named constructor to `std::simd`

Published Proposal, 2025-06-13

This version:

<http://wg21.link/P3440R1>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to add `std::simd_mask::n_elements` named constructor to create a mask containing an exact number of set bits. Such a function is notably useful for handling loop remainders.

Table of Contents

- 1 Revision History**
- 2 Motivation**
- 3 Implementation experience**
- 4 Wording**
 - 4.1 Modify [`simd.mask.overview`]
 - 4.2 Add a new section for `basic_simd_mask` named constructors [`simd.mask.named_ctor`]

§ 1. Revision History

R0 => R1

- Freshened up the wording to match the current state of the draft proposal.

§ 2. Motivation

When iterating over large dynamic data sets using `simd` ([P1928R15]) loop, there will inevitably be situations where the very last block of data doesn't fill the entire `simd` object. This remainder needs to be processed using a partially filled `simd` object. For example:

```
void fn(float* ptr, std::size_t count)
{
    // Process complete SIMD blocks.
    auto wholeBlocks = count / simd<float>::size();
    for (int i=0; wholeBlocks; ++i)
    {
        auto block = simd<float>(i * simd<float>::size());
        process(block); // Process an entire simd-worth of data.
    }

    // Process the remainder.
    auto remainder = count % simd<float>::size();
    if (remainder > 0)
    {
        simd_mask<float> remainderMask ([=count](auto idx) { return idx < count; >})
        auto remainderBlock =
            simd_load<simd<float>>(ptr + (count - remainder), remainder, simd_default_
            process(remainderBlock, remainderMask); // Do the work on part of the SIMD o
    }
}
```

In this example the remainder has been handled by creating a mask in which only the bits `[0..remainder)` are active. Note that the partial load has been handled using the `partial_load(range)` function described in [P1928R15] which is memory-safe and can be efficiently implemented. However, the processing itself is taking a `simd` and only operating on the subset of its elements which correspond to the remainder, and for this processing a suitable remainder mask must be generated.

In the example the remainder mask has been created using a mask generator, where each bit in the mask is created using a comparison against the number of required bits. There are other ways of creating that mask, three variants of which are illustrated here:

```
int numRemainderBits = ...;

// Use an iota object with a comparison. This is quite compact, but will have so
// runtime conversion to deal with the `float` comparison.
auto remainder1 = iota<simd<float>> < numRemainderBits;

// Like the previous, but explicitly avoid the runtime conversion to float.
auto tmp = iota<simd<uint32_t>> < numRemainderBits;
auto remainder2 = simd_mask<float>(tmp); // Convert to the correct type of mask.

// Use the facilities of new constructors [[P2876R2]] to build a mask from an
// integer bit set. This generates efficient code on compact mask
// machines (e.g., Intel AVX-512, AVX-10). It doesn't handle masks containing mo
// 64 elements without a change in scalar integer type.
auto m = (uint64_t(1) << numRemainderBits) - 1;
auto remainder3 = simd_mask<float>(m);
```

One serious issue with this selection of methods is that there is no single obvious style to use to generate the best code across a range of targets. For example, the last method works well on compact-mask targets (e.g., Intel AVX-512), but poorly on wide-mask targets (e.g., Intel SSE). Adding conditional code around the mask to reflect on the target and generate the mask differently just leads to a reduction in portability and an increase in verbosity.

Manual mask generation can introduce subtle issues for corner cases. For example, constructing from a compact integer as in the last code snippet above will work only if the integer itself is constructed properly. If the integer type was too small (e.g., `uint16_t` for a 64-bit mask) it may silently fail for some targets. Or in a wide-mask variant in which the mask is generated using the comparison `iota<simd<uint8_t>> < n` this will fail if `simd<uint8_t>` has more than 256 elements, leading to portability issues for targets which support more elements than this.

To avoid the issues with manual mask generation we propose that a named constructor is provided which populates a `simd_mask` with exactly N bits active at positions `[0..N)`. By making this function part of `simd` itself the implementation can choose the most efficient implementation for the target, and it can correctly handle all possible cases:

```
static constexpr basic_simd_mask basic_simd_mask::n_elements(simd_size_type coun
```

Given a count of zero, an empty mask will be returned. When the count is in the range `[0..count)` a mask containing just that many bits will be returned. When the count is larger than the mask, a mask with all bits

set will be returned.

§ 3. Implementation experience

Intel's implementation of `simd` has had this named constructor since very early on, and it is used throughout our example code base. It makes generating efficient mask remainders across all Intel targets efficient and easy, and it makes the code's intent very obvious.

§ 4. Wording

The following wording is a diff against the current draft standard.

§ 4.1. Modify `[simd.mask.overview]`

Add the new named constructor in its own section after the list of constructors:

```
// [simd.mask.ctor], basic_simd_mask constructors
constexpr explicit basic_simd_mask(value_type) noexcept;
template<size_t UBytes, class UAbi>
    constexpr explicit basic_simd_mask(const basic_simd_mask<UBytes, UAbi>&)
template<class G> constexpr explicit basic_simd_mask(G&& gen) noexcept;

// [simd.mask.named_ctor] basic_simd_mask named constructors
static constexpr basic_simd_mask n_elements(simd-size-type count) noexcept
```

§ 4.2. Add a new section for `basic_simd_mask` named constructors `[simd.mask.named_ctor]`

`basic_simd_mask` named constructors `[simd.mask.named_ctor]`

```
static constexpr basic_simd_mask n_elements(simd-size-type count) noexcept;
```

Returns:

A `basic_simd_mask` object where the i^{th} element is initialized to `i < count` for all `i` in the range of `[0..size)`.